



Bluetooth LE-PHY6222

Version: 20240613

Contents

1	Hardware environment	2
1.1	Datasheet	2
1.2	Development board	2
1.3	Peripherals	2
2	Software environment	5
2.1	Keil	5
3	Develop software	6
3.1	Enable scanning	6
3.2	Flash and RAM footprint	6
4	Firmware flashing	10
4.1	Wire connection	10
4.2	Firmware description	10
4.3	Flashing method	10
5	Features	13
5.1	Startup time	13
5.2	Flash partition	15
5.3	Power consumption	17
5.4	RTC precision	18



This topic describes the basics of the chip platform used in TuyaOS development. For more information, visit [Tuya Developer Forum](#).

1 Hardware environment

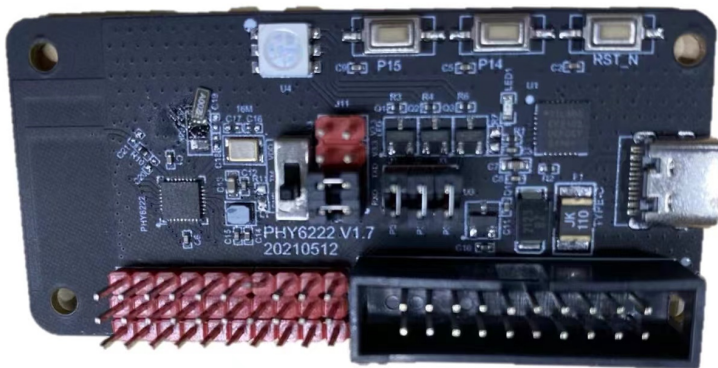
1.1 Datasheet

The datasheet can be found in `hardware\PHY6222\chip_manual\PHY6222_BLE_SoC_Datasheet_v1.2_20210430.pdf`.

1.2 Development board

Phyplus official PHY6222 development board is used. You can purchase the board from Phyplus.

Or, purchase it (**part number: 1.15.17.00173**) from the [Tuya Developer Platform](#).



1.3 Peripherals

UART

This platform has two UARTs. UART0 is used for log output. UART1 is used for flashing, authorization, and debugging.

UART	Feature	Pin	Remarks
UART0	TX	P2	Log

UART	Feature	Pin	Remarks
UART0	RX	P0	
UART1	TX	P9	Flashing, authorization, and testing.
UART1	RX	P10	

SPI

This platform has two SPI buses.

SPI	Feature	Pin
SPI0	CS	P32
SPI0	CLK	P31
SPI0	SDI	P18
SPI0	SDO	P20
SPI1	CS	P14
SPI1	CLK	P15
SPI1	SDI	P33
SPI1	SDO	P34

I2C

This platform has two I2Cs. To display SDK information (TuyaOS demo), the OLED is pre-initialized. Therefore, only one I2C can be used for testing.

I2C	Feature	Pin
I2C0	SCL	P24
I2C0	SDA	P23

PWM

Channel	Pin
0	P31
1	P32
2	P26
3	P33
4	P34
5	P7

ADC

Only support 12-bit resolution.

Voltage value is returned.

Channel	Pin
0	P11
1	P23
2	P24
3	P14
4	P15
5	P20

Key control

P7

- Press the button to wake up the device.
- Press and hold the button for three seconds to factory reset the device.

Power-on detection pin

P1

Set this pin to high after entering the `main` function.

2 Software environment

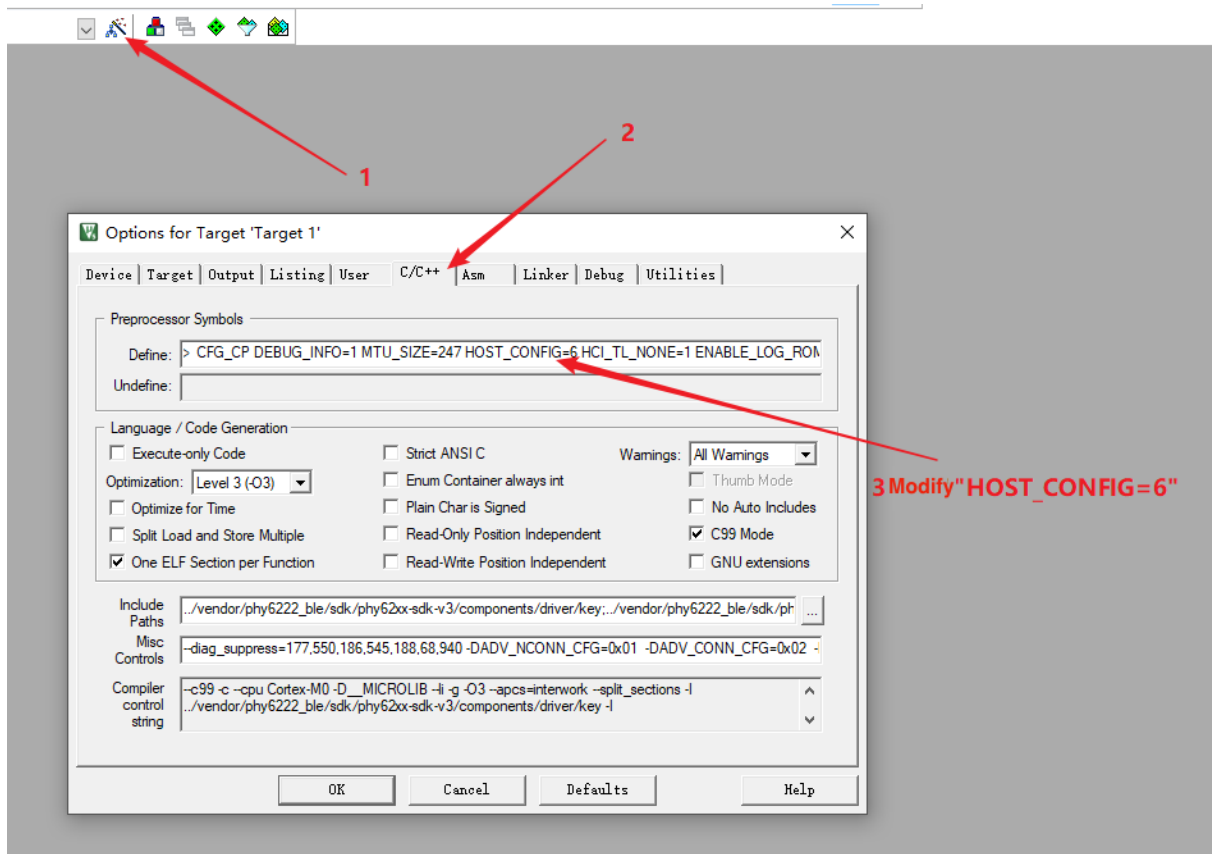
2.1 Keil

1. Download Keil from the [Arm Keil](#) website. **mdk528a.exe** is recommended. During installation, keep all the default settings.
2. If you are prompted to install a driver, continue with **Install**.
3. When opening Keil, if you are prompted to install a software pack, close the window.
4. After Keil is installed, add the path of the [UV4.exe](#) file to the **environment variable**. For example: `TUYAOS_COMPILE_TOOL: C:\Keil_v5\UV4`, as shown below.

Path	C:\Users\su\AppData\Local\Programs\Python\Python38\Scrip...
TEMP	C:\Users\su\AppData\Local\Temp
TMP	C:\Users\su\AppData\Local\Temp
TUYAOS_COMPILE_TOOL	C:\Keil_v5\UV4

3 Develop software

3.1 Enable scanning



```
1 HOST_CONFIG=6
```

3.2 Flash and RAM footprint

1. Disable the testing feature.

```
1 #define TUYA_SDK_TEST 0
```

2. Disable logging.


```
1 #define ENABLE_LOG 0
2 #define BOARD_ENABLE_LOG 0
```

3. Disable scanning.

```
1 HOST_CONFIG=4
```

4. After a full compilation of the whole project, open the .map file. You will get the following screen.

- The flash footprint:

```
=====
Total RO Size (Code + RO Data)          97792 ( 95.50kB)
Total RW Size (RW Data + ZI Data)       28872 ( 28.20kB)
Total ROM Size (Code + RO Data + RW Data) 98352 ( 96.05kB)
=====
```

- The RAM footprint:

```
0x1fffa93b COMPRESSED 0x00000001 PAD
0x1fffa93c - 0x00000010 Zero RW 8554 .bss rf.lib(phy_sec_ext.o)
0x1fffa94c - 0x000000a0 Zero RW 8649 .bss ble_host.lib(l2cap_util.o)
0x1fffa9ec - 0x0000000f Zero RW 8714 .bss ble_host.lib(sm_mgr.o)
0x1fffa9fb COMPRESSED 0x00000005 PAD
0x1fffaa00 - 0x00001000 Zero RW 7606 STACK startup_armcm0.o

Load Region LR_ROM_JT_GC (Base: 0x1fff0000, Size: 0x00000800, Max: 0x00000800, ABSOLUTE, COMPRESSED[0x000040c])

Execution Region JUMP_TABLE (Exec base: 0x1fff0000, Load base: 0x1fff0000, Size: 0x0000400, Max: 0x0000400, ABSOLUTE)

Exec Addr Load Addr Size Type Attr Idx E Section Name Object
```

- The free RAM:
 - = 0x1fffffff – (0x1fffaa00 + 0x00001000)
 - = 17919 bytes
 - = 17.5 KB

When you enable scanning

```
1 HOST_CONFIG=6
```

After a full compilation of the whole project, open the `.map` file. You will get the following screen.

- The flash footprint:

```
=====
Total RO   Size (Code + RO Data)          110968 ( 108.37kB)
Total RW   Size (RW Data + ZI Data)         30948 (  30.22kB)
Total ROM  Size (Code + RO Data + RW Data)  111544 ( 108.93kB)
=====
```

Figure 1: img

- The RAM footprint:

```
0x1fffcdf7 COMPRESSED 0x00000001 PAD
0x1fffcdf8 - 0x00000010 Zero RW 8594 .bss rf.lib(phy_sec_ext.o)
0x1fffce08 - 0x000000a0 Zero RW 8707 .bss ble_host.lib(l2cap_util.o)
0x1fffcea8 - 0x0000000f Zero RW 8772 .bss ble_host.lib(sm_mgr.o)
0x1fffceb7 COMPRESSED 0x00000001 PAD
0x1fffceb8 - 0x00001000 Zero RW 7611 STACK startup_armcm0.o

Load Region LR_ROM_JT_GC (Base: 0x1fff0000, Size: 0x00000800, Max: 0x00000800, ABSOLUTE, COMPRESSED[0x0000040c])

Execution Region JUMP_TABLE (Exec base: 0x1fff0000, Load base: 0x1fff0000, Size: 0x00000400, Max: 0x00000400, ABSOLUTE)

Exec Addr  Load Addr  Size      Type  Attr  Idx  E Section Name  Object
```

Figure 2: img

The free RAM:

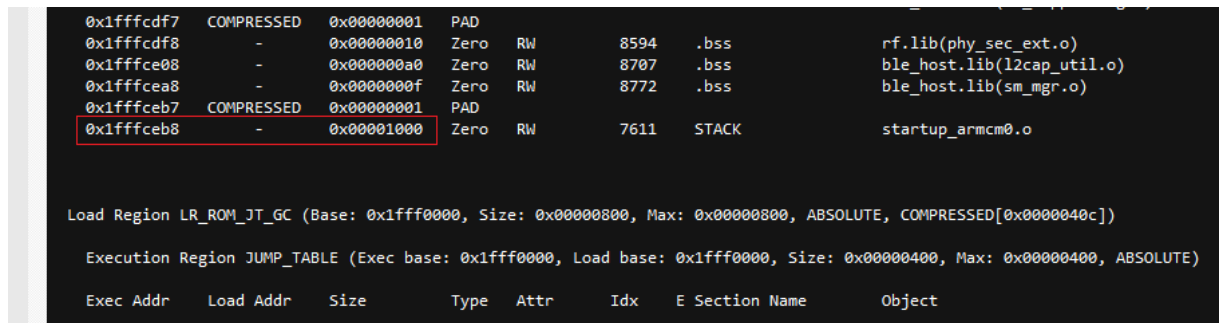
- = $0x1fffffff - (0x1fffceb8 + 0x00001000)$
- = 8519 bytes
- = 8.3 KB

Calculate free RAM

Some Phyplus code might be stored in RAM. Therefore, The free RAM we calculate may vary from the one provided in the `.map` file. Here is how to calculate the free

RAM.

1. Open the `.map` file.
2. Find the `Memory Map of the image` section.
3. Navigate to the end of the section and check the last line, as shown below.



0x1fffcdf7	COMPRESSED	0x00000001	PAD						
0x1fffcdf8	-	0x00000010	Zero	RW	8594	.bss		rf.lib(phy_sec_ext.o)	
0x1fffce08	-	0x000000a0	Zero	RW	8707	.bss		ble_host.lib(l2cap_util.o)	
0x1fffcea8	-	0x0000000f	Zero	RW	8772	.bss		ble_host.lib(sm_mgr.o)	
0x1fffceb7	COMPRESSED	0x00000001	PAD						
0x1fffceb8	-	0x00001000	Zero	RW	7611	STACK		startup_armcm0.o	

Load Region LR_ROM_JT_GC (Base: 0x1fff0000, Size: 0x00000800, Max: 0x00000800, ABSOLUTE, COMPRESSED[0x0000040c])

Execution Region JUMP_TABLE (Exec base: 0x1fff0000, Load base: 0x1fff0000, Size: 0x00000400, Max: 0x00000400, ABSOLUTE)

Exec Addr	Load Addr	Size	Type	Attr	Idx	E	Section Name	Object
-----------	-----------	------	------	------	-----	---	--------------	--------

The free RAM is calculated by: $0x1fffffff - (0x1fffceb8 + 0x00001000)$

Quickly locate: `LR_ROM_JT_GC`

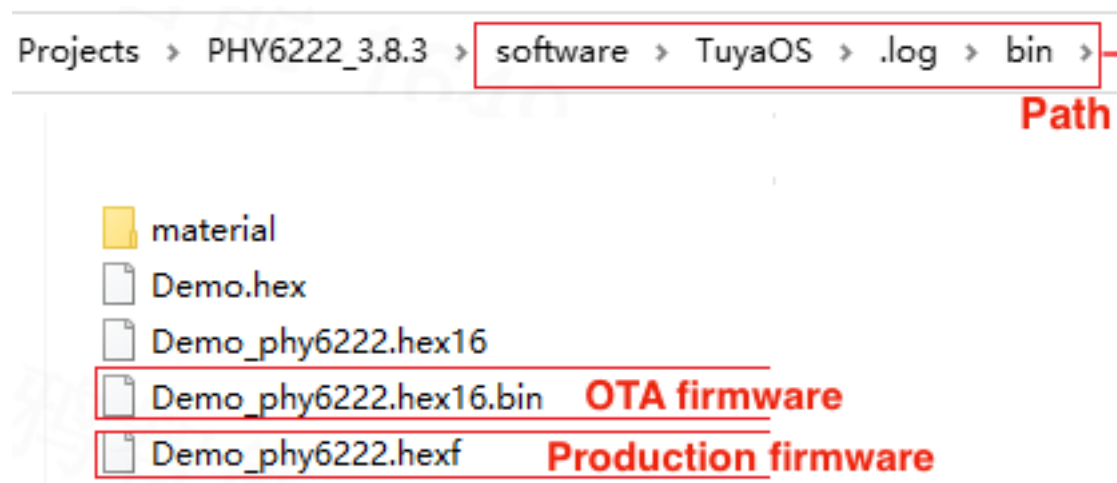
4 Firmware flashing

4.1 Wire connection

To flash firmware to the device, connect the device and the USB to TTL converter as described below.

USB to TTL converter	Device
VCC	VCC
GND	GND
TX	RX
RX	TX

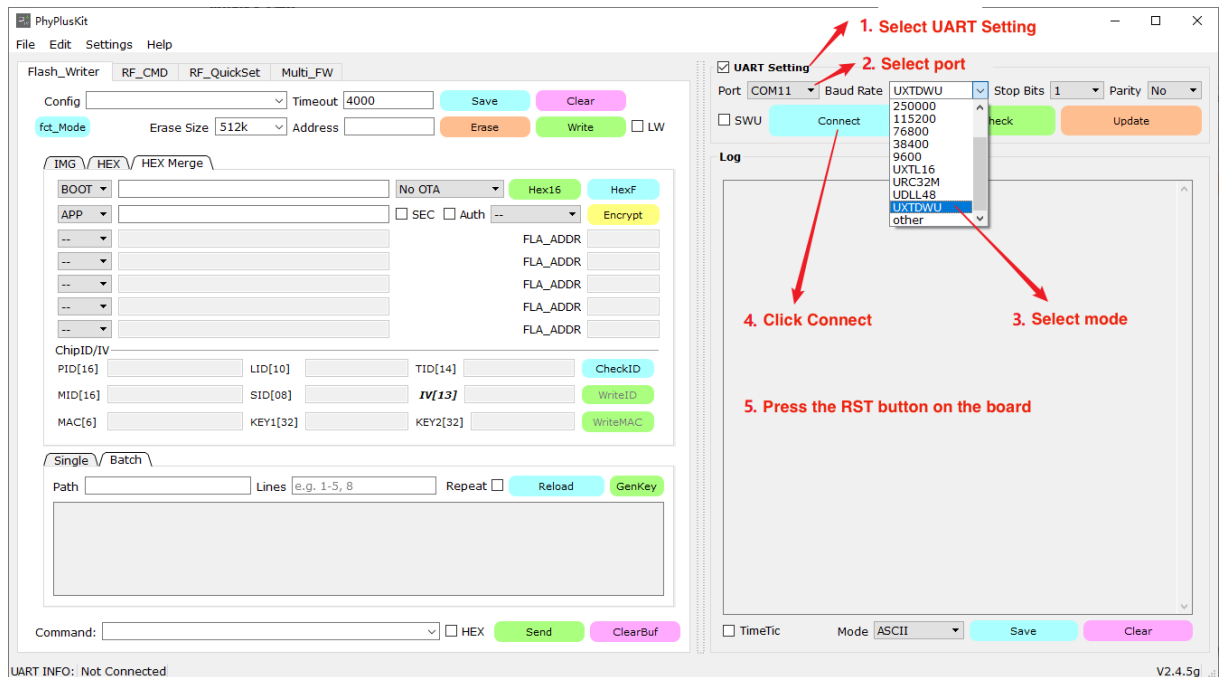
4.2 Firmware description



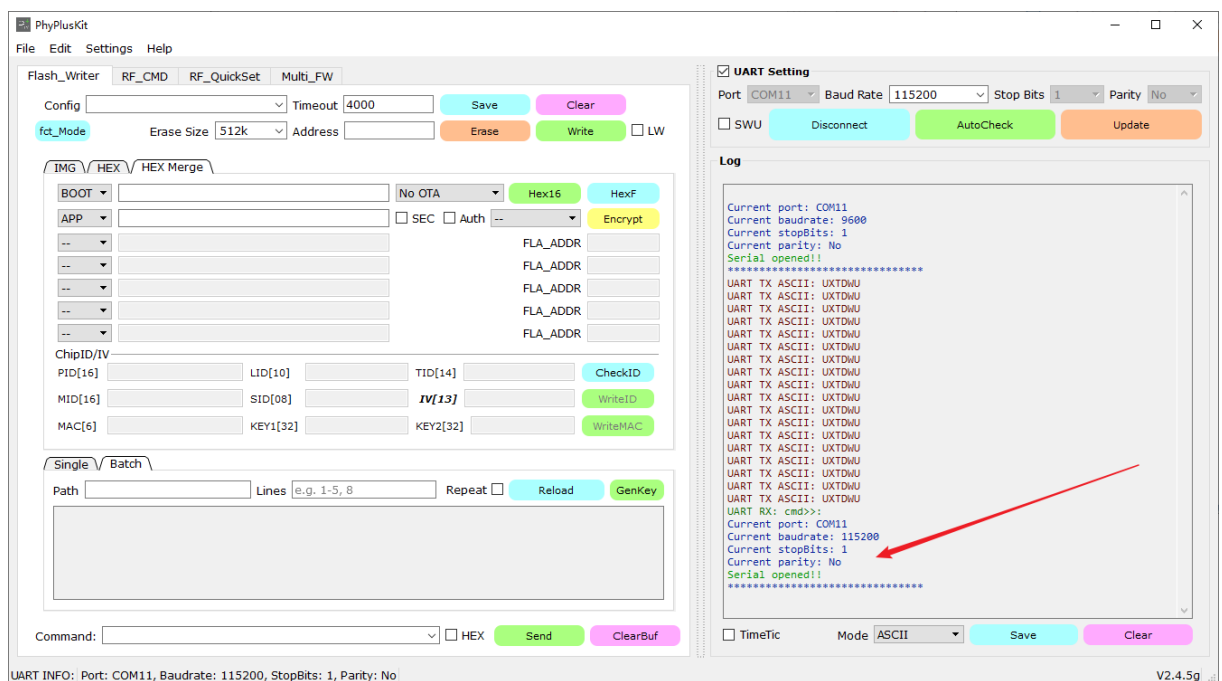
4.3 Flashing method

Open the flashing tool from `pc\tools\PHY6222`.

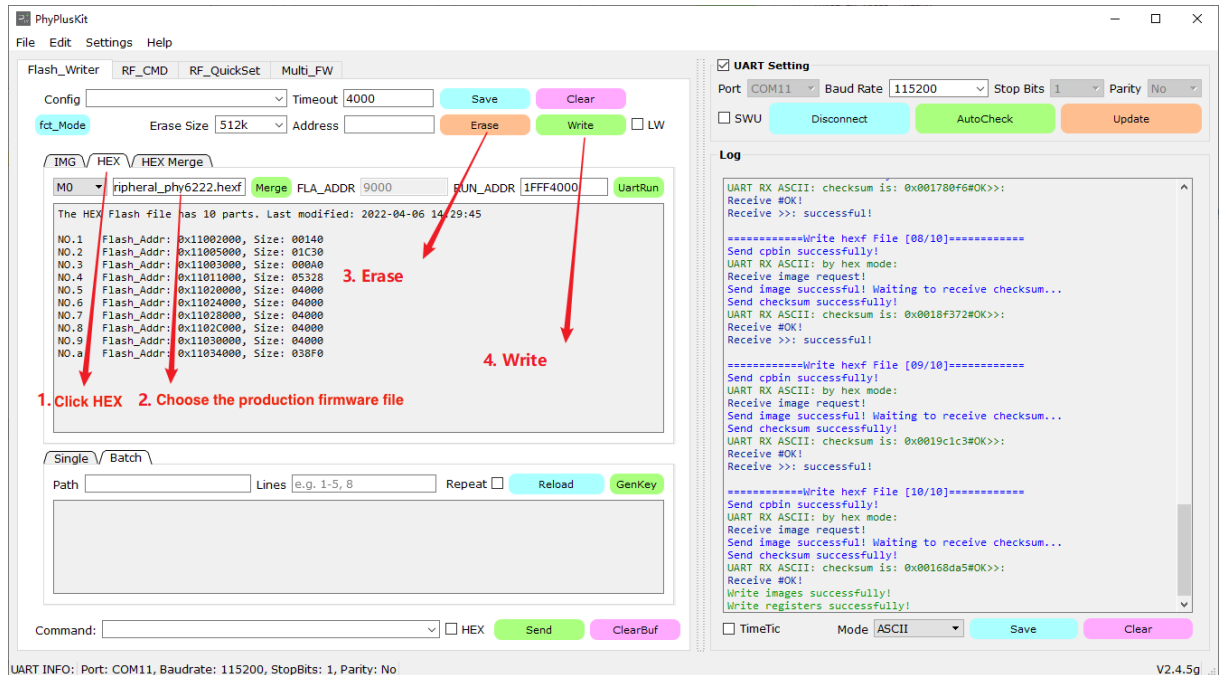
1. Connect the device to the computer through a USB to TTL converter.



2. Serial port is connected.



3. Import the firmware binary file and proceed with erasing and writing.



5 Features

5.1 Startup time

- Use a logic analyzer to measure the signal time difference between the **VCC pin** and the **startup time detection pin**.
- Detection pins: peripheral → startup time detection pin

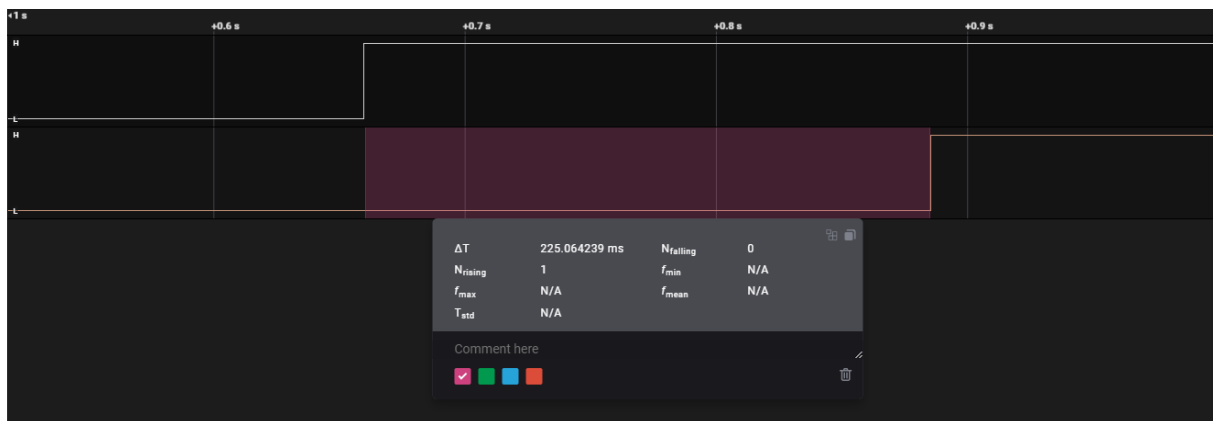


Figure 3: img

5.2 Flash partition

	OTA	172K
0x55000	OTA info	4K
0x54000	tuya_ble_sdk_info	16K
0x50000	unused	80K
0x3C000	App	172K
0x11000	Resource	32K
0x09000	SLB Bootloader	16K
0x05000	FCDS	4K
0x04000	2nd Boot info	4K
0x03000	1st Boot info	4K
0x02000	Reserved By PHYPlus	8K
0x00000		512K

Boot ROM. Protocol stack. Common peripheral drivers. ATE AT command.	96K
---	-----

Adjust flash partitions

In the original project, you can modify the following macros to adjust the default flash partitions.

```
1 // Flash
2 #ifndef BOARD_FLASH_BULK_DATA_START_ADDR
3 #define BOARD_FLASH_BULK_DATA_START_ADDR (0x4C000)
4 #endif
5 #ifndef BOARD_FLASH TUYA_INFO_START_ADDR
6 #define BOARD_FLASH TUYA_INFO_START_ADDR (0x50000)
7 #endif
8 #ifndef BOARD_FLASH_OTA_INFO_ADDR
9 #define BOARD_FLASH_OTA_INFO_ADDR (0x54000)
10 #endif
11 #ifndef BOARD_FLASH_OTA_START_ADDR
12 #define BOARD_FLASH_OTA_START_ADDR (0x55000)
13 #endif
14 #ifndef BOARD_FLASH_OTA_END_ADDR
15 #define BOARD_FLASH_OTA_END_ADDR (0x80000)
16 #endif
17 #ifndef BOARD_FLASH_OTA_SIZE
18 #define BOARD_FLASH_OTA_SIZE (BOARD_FLASH_OTA_END
19 _ADDR - BOARD_FLASH_OTA_START_ADDR)
20 #endif
```

To change the OTA memory location, besides editing the above macro, you also need to edit the bootloader project accordingly.

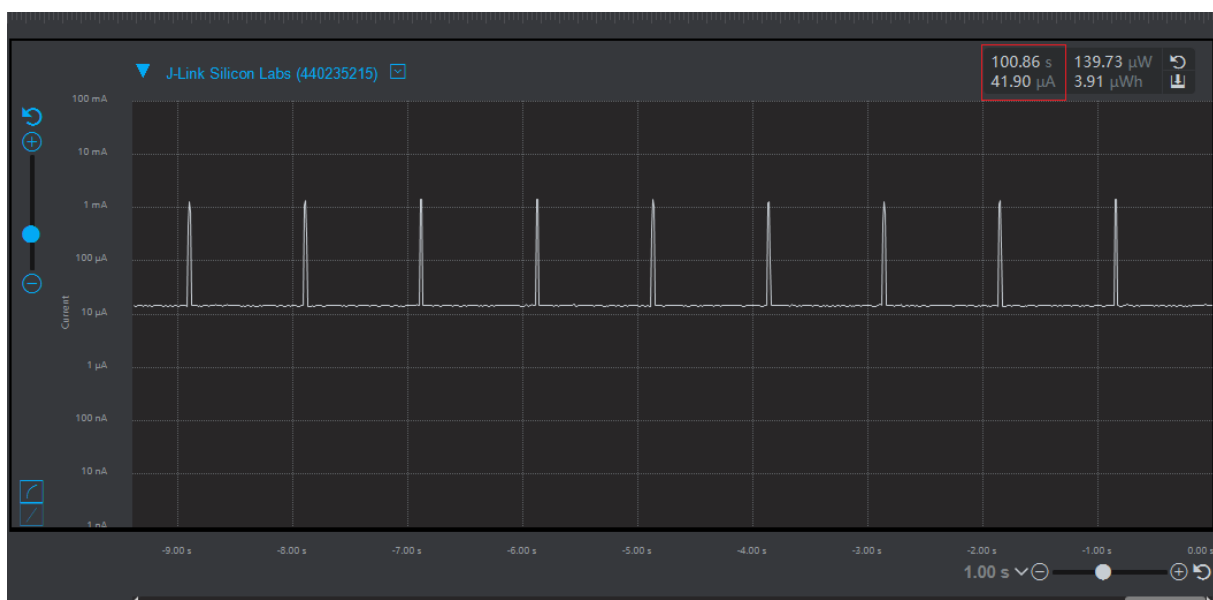
1. Open the bootloader project from `\vendor\phy6222_ble\sdk\phy62xx-sdk-v3\example\OTA\slboot`.
2. Edit `SLB_EXCH_AREA_BASE` and `SLB_EXCH_AREA_SIZE` and then compile the bootloader project.

```
slb.h
1 #ifndef __SLB_H
2 #define __SLB_H
3
4
5 #ifndef SLB_EXCH_AREA_BASE
6     #define SLB_EXCH_AREA_BASE 0x11055000
7 #endif
8 #ifndef SLB_EXCH_AREA_SIZE
9     #define SLB_EXCH_AREA_SIZE (172*1024) //172K
10 #endif
11
12 #define SLB_OOB_PIECE_SIZE_MAX 256
13
```

5.3 Power consumption

Measure the average power consumption of typical one-second advertising.

Power consumption in sleep mode is 13 μA .



5.4 RTC precision

- Normal operation mode: The time drift is less than two seconds within 20 hours.
- Sleep mode: The time drift is about eight seconds within in 11 hours.